

NOX — TECHNICAL ARCHITECTURE REPOSITORY

Urban Entertainment Infrastructure Platform

Engineering System Design & Repository Blueprint

1. SYSTEM OVERVIEW

Product Classification

NOX is architected as:

- mobile-first platform
- vertical SaaS infrastructure
- marketplace ecosystem
- real-time event platform
- fintech-adjacent system

Core technical requirements:

- scalability
 - low operational overhead
 - high concurrency support
 - real-time updates
 - fraud-resistant ticketing
 - mobile-first performance
 - rapid deployment cycles
-

2. HIGH-LEVEL ARCHITECTURE

System Components

Frontend Layer

Consumer Mobile App

Framework: Flutter

Responsibilities:

- event discovery
 - ticket purchasing
 - QR ticket storage
 - user profiles
 - notifications
-

Organizer Dashboard

Framework: Next.js

Responsibilities:

- event management
 - analytics
 - ticket tracking
 - guest list management
-

Venue Dashboard

Framework: Next.js

Responsibilities:

- venue analytics
 - attendance metrics
 - event scheduling
-

Admin Panel

Framework: Next.js

Responsibilities:

- moderation
 - dispute management
 - promoter verification
 - operational monitoring
-

Backend Layer

API Gateway

Recommended: NestJS

Responsibilities:

- authentication
 - business logic
 - API orchestration
 - rate limiting
 - permissions
-

Database

Recommended: PostgreSQL (Supabase)

Responsibilities:

- relational data
 - real-time subscriptions
 - row-level security
 - analytics querying
-

Storage Layer

Recommended: Supabase Storage

Used for:

- event banners
 - venue photos
 - user avatars
 - ticket assets
-

Real-Time Infrastructure

Recommended: Supabase Realtime

Used for:

- live ticket updates
 - event changes
 - attendance tracking
 - dashboard refreshes
-

Payment Layer

Initial Strategy: External wallet integrations.

Providers:

- KBZPay
 - WavePay
 - AyaPay
 - CBPay
-

Notification Layer

Recommended: Firebase Cloud Messaging

Used for:

- event reminders
 - payment confirmations
 - ticket updates
 - promoter announcements
-

3. REPOSITORY STRUCTURE

```
nox-platform/  
|  
├─ apps/  
|   ├─ mobile-app/  
|   ├─ promoter-dashboard/  
|   ├─ venue-dashboard/  
|   ├─ admin-panel/  
|   └─ qr-scanner/  
|  
└─ backend/
```

```
├── api-gateway/
│   ├── auth-service/
│   ├── payment-service/
│   ├── notification-service/
│   ├── analytics-service/
│   └── ticket-service/
├── packages/
│   ├── ui-kit/
│   ├── shared-types/
│   ├── shared-utils/
│   └── design-tokens/
├── infrastructure/
│   ├── docker/
│   ├── terraform/
│   ├── kubernetes/
│   └── nginx/
├── docs/
│   ├── api-docs/
│   ├── architecture/
│   ├── onboarding/
│   └── security/
├── scripts/
├── .github/
├── package.json
├── turbo.json
└── README.md
```

4. MONOREPO STRATEGY

Recommended Tool

Turborepo

Why:

- shared packages
- scalable architecture
- unified builds
- deployment efficiency

- shared UI systems
-

5. FRONTEND ARCHITECTURE

Mobile App

Framework

Flutter

Architecture Pattern

Feature-first modular architecture.

Flutter Folder Structure

```
lib/  
|  
├── core/  
│   ├── constants/  
│   ├── theme/  
│   ├── network/  
│   ├── storage/  
│   └── utils/  
|  
├── features/  
│   ├── auth/  
│   ├── events/  
│   ├── tickets/  
│   ├── venues/  
│   ├── profile/  
│   └── notifications/  
|  
├── shared/  
│   ├── widgets/  
│   ├── models/  
│   └── services/  
└── main.dart
```

State Management

Recommended: Riverpod

Why:

- scalability
 - testability
 - clean architecture support
 - async handling
-

Navigation

Recommended: GoRouter

Local Storage

Recommended: Hive

Used for:

- cached tickets
 - offline QR validation
 - saved events
-

6. BACKEND ARCHITECTURE

Core Backend Stack

API Framework

NestJS

Why:

- modular architecture
- enterprise scalability
- TypeScript support
- guards/interceptors
- microservice compatibility

API Pattern

REST initially.

Future: GraphQL gateway possible.

Authentication

Recommended: JWT + Refresh Tokens

Providers:

- Phone OTP
 - Google OAuth
 - Apple OAuth
-

Example Auth Flow

```
Client → OTP Request
Client → Verify OTP
Server → Generate Access Token
Server → Generate Refresh Token
Client → Store Securely
```

7. DATABASE ARCHITECTURE

PostgreSQL Schema Design

Core Principles

- normalized structure
 - relational integrity
 - audit logging
 - scalable analytics queries
-

Key Tables

users

```
CREATE TABLE users (  
  id UUID PRIMARY KEY,  
  username TEXT,  
  email TEXT,  
  phone TEXT,  
  role TEXT,  
  created_at TIMESTAMP  
);
```

venues

```
CREATE TABLE venues (  
  id UUID PRIMARY KEY,  
  name TEXT,  
  address TEXT,  
  latitude DOUBLE PRECISION,  
  longitude DOUBLE PRECISION,  
  verified BOOLEAN  
);
```

events

```
CREATE TABLE events (  
  id UUID PRIMARY KEY,  
  organizer_id UUID,  
  venue_id UUID,  
  title TEXT,  
  description TEXT,  
  start_time TIMESTAMP,  
  end_time TIMESTAMP,  
  genre TEXT,  
  status TEXT  
);
```

tickets

```
CREATE TABLE tickets (  
  id UUID PRIMARY KEY,  
  event_id UUID,  
  user_id UUID,  
  qr_code TEXT,  
  payment_status TEXT,  
  checked_in BOOLEAN  
);
```

8. QR VALIDATION SYSTEM

Ticket Generation Flow

```
Payment Success  
  ↓  
Generate Ticket ID  
  ↓  
Encrypt Payload  
  ↓  
Generate QR Code  
  ↓  
Store Validation Hash  
  ↓  
Deliver Ticket
```

QR Payload Structure

```
{  
  "ticket_id": "uuid",  
  "event_id": "uuid",  
  "issued_at": "timestamp",  
  "checksum": "encrypted_signature"  
}
```

Validation Logic

Scanner verifies:

- authenticity
 - ticket status
 - duplicate scans
 - event ownership
 - expiration rules
-

9. PAYMENT ARCHITECTURE

MVP PAYMENT MODEL

DO NOT build wallet infrastructure initially.

Use:

- deep links
 - redirect verification
 - transaction confirmation
-

Payment Flow

```
Select Ticket
  ↓
Choose Wallet
  ↓
Redirect to Payment App
  ↓
Payment Completed
  ↓
Webhook Verification
  ↓
Ticket Issued
```

Security Measures

Required

- webhook validation
 - signed callbacks
 - replay attack prevention
 - transaction reference checks
-

10. ANALYTICS ARCHITECTURE

Product Analytics

Recommended: PostHog

Track:

- retention
 - event discovery behavior
 - ticket conversion
 - purchase funnels
 - promoter engagement
-

Business Analytics

Recommended: Metabase

Used for:

- venue reporting
 - revenue analytics
 - promoter dashboards
-

11. INFRASTRUCTURE STACK

Cloud Infrastructure

MVP Phase

Hosting

- Vercel (Dashboards)
 - Supabase Cloud
 - Firebase Messaging
-

Scaling Phase

Move toward:

- AWS
 - Kubernetes
 - dedicated PostgreSQL clusters
-

Containerization

Docker

Example:

```
FROM node:20-alpine
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build
EXPOSE 3000
CMD ["npm", "run", "start"]
```

12. DEVOPS PIPELINE

CI/CD

Recommended

GitHub Actions

Deployment Pipeline

```
Push to GitHub
  ↓
Run Tests
  ↓
Run Linting
  ↓
Build Artifacts
  ↓
Deploy Preview
  ↓
Production Deployment
```

Example Workflow

```
name: Deploy

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3
```

- run: npm install
 - run: npm run build
-

13. SECURITY ARCHITECTURE

Authentication Security

Required:

- JWT rotation
 - OTP expiration
 - rate limiting
 - device fingerprinting
-

API Security

Required:

- request validation
 - RBAC permissions
 - input sanitization
 - throttling
-

Fraud Prevention

Required:

- duplicate QR detection
 - suspicious transaction detection
 - event abuse monitoring
-

14. SCALABILITY STRATEGY

Initial Scale Targets

Supports:

- 50K users
 - 10K MAU
 - 500+ monthly events
 - concurrent ticket launches
-

Scaling Strategy

Horizontal Scaling

Future migration:

- Kubernetes
 - load balancing
 - read replicas
 - caching layers
-

Caching Strategy

Recommended: Redis

Used for:

- event feeds
 - trending events
 - ticket inventory caching
-

15. API DESIGN

Example REST Endpoints

Auth

```
POST /auth/login
POST /auth/verify-otp
POST /auth/refresh
```

Events

```
GET /events
GET /events/:id
POST /events
PATCH /events/:id
```

Tickets

```
POST /tickets/purchase
GET /tickets/me
POST /tickets/validate
```

16. OBSERVABILITY STACK

Monitoring

Recommended:

- Sentry
 - Grafana
 - Prometheus
-

Logging

Recommended: Pino

Centralized via:

- Loki OR
 - ELK Stack
-

17. ENGINEERING STANDARDS

Code Standards

TypeScript Everywhere

Required for:

- backend
 - dashboards
 - shared types
-

Branching Strategy

```
main
staging
develop
feature/*
hotfix/*
```

Commit Convention

```
feat:
fix:
refactor:
```

chore:
docs:

18. RECOMMENDED MVP ENGINEERING TEAM

Technical Roles

Lead Engineer

System architecture.

Flutter Engineer

Mobile development.

Backend Engineer

APIs + infrastructure.

UI Engineer

Dashboard systems.

DevOps Engineer

CI/CD + infrastructure.

19. FUTURE TECH EXPANSION

Phase 2 Technical Layers

AI Recommendation Engine

Used for:

- personalized events
 - demand prediction
 - promoter optimization
-

Dynamic Pricing Engine

Used for:

- surge pricing
 - VIP optimization
 - occupancy balancing
-

Fintech Expansion

Potential:

- stored wallets
 - loyalty points
 - nightlife BNPL
 - creator payouts
-

20. FINAL TECHNICAL THESIS

The architecture should prioritize:

1. Speed of deployment
2. Operational simplicity
3. Scalability readiness
4. Fraud resistance
5. Mobile performance
6. Real-time infrastructure

The correct strategic approach is:

“Lean infrastructure with enterprise scalability pathways.”

This avoids:

- overengineering
- unnecessary cloud costs
- premature complexity

while preserving the ability to scale into a regional entertainment infrastructure platform.

FINAL RECOMMENDED STACK

Frontend

- Flutter
- Next.js
- Tailwind

Backend

- NestJS
- Supabase/PostgreSQL
- Redis

Infrastructure

- Docker
- GitHub Actions
- Vercel
- Cloudflare

Analytics

- PostHog
- Metabase

Monitoring

- Sentry
- Grafana

Notifications

- Firebase Cloud Messaging

Payments

- Local wallet integrations

Scalability Philosophy

Lean MVP → Regional Infrastructure Platform